Dynamic UAV Routing: A Controlled Comparison of Naive Dijkstra Replanning and Reinforcement Learning

*Simra Imran
School of Electrical Engineering and Computer Science (SEECS)
National University of Sciences and Technology (NUST)
Email: simraimran158@gmail.com

*Muzzammil Sajid
Faculty of Mechanical Engineering (FME)
Ghulam Ishaq Khan Institute of Engineering Sciences and Technology (GIKI)
Email: muzzammilsajid1@gmail.com

***Abstract*—Unmanned aerial vehicles (UAVs) operating in dynamic environments must route around obstacles that can appear, disappear, or move during flight. Classical graph-search methods provide optimal paths on a known static graph but must be recomputed when the graph changes; reinforcement learning (RL) offers an alternative policy-based approach that reacts through learned action selection rather than explicit search. This paper presents a controlled, paired comparison of two classical replanning baselines — naive Dijkstra replanning and A* replanning — against a Double DQN policy trained with Hindsight Experience Replay (HER) and curriculum learning, using a shared 15×15 grid-world UAV routing environment with identical movement and obstacle rules across all three methods. We first validate the RL policy on a static 40-scenario evaluation (obstacle density 0.2), where Dijkstra succeeds on 100% of scenarios against 97.5% for DQN, with a mean path-cost penalty of 0.92 units above optimal and a ∼3.3× compute-time overhead for DQN, both figures independently reproduced for this revision. We then evaluate all three methods on a paired 50-scenario dynamic evaluation with three shared toggling obstacles. Dijkstra and A* both succeed on all 50 scenarios and, as expected for two optimal planners, produce identical path costs; RL succeeds on 49 of 50 and produces significantly higher-cost paths than both classical planners (Wilcoxon signed-rank $W = 0$, $p = 5.64 \times 10^{-7}$ vs. Dijkstra). The compute-time picture is more nuanced than a single Dijkstra baseline suggests: naive Dijkstra replanning shows no significant compute-time advantage over RL ($p = 0.063$), but A*'s heuristic-guided search resolves the same replanning events roughly 16× faster than Dijkstra ($p = 3.6 \times 10^{-15}$) at identical path quality, revealing that the earlier "RL vs. classical" framing conflated a naive-implementation gap with a genuine algorithmic one. These results support an honest trade-off framing: RL demonstrates substantial adaptability but does not match either classical planner in optimality, reliability, or compute cost at the tested scale, and a well-implemented classical planner (A*) is a stronger deployment baseline than naive Dijkstra replanning alone. We report all evaluations on a single trained policy (seed 42) and treat the absence of multi-seed training variance as an explicit, unresolved limitation rather than an omission.**

***Index Terms*—UAV routing, Dijkstra algorithm, reinforcement learning, dynamic obstacles, path planning, grid world**

*These authors contributed equally to this work.

I. Introduction

Unmanned aerial vehicles (UAVs) increasingly operate in environments where routing conditions can change during flight. Obstacles may appear or disappear, restricted regions may change, and the route selected at launch may no longer be valid later in the mission. This makes dynamic path planning a central problem for autonomous UAV navigation [1], [2].

Classical graph-search algorithms remain a strong baseline for this problem because they are deterministic, interpretable, and optimal when the graph and edge costs are known. Dijkstra's algorithm, in particular, returns a shortest path when all edge weights are non-negative. However, in a dynamic environment, each obstacle change can invalidate the previous path and require replanning from the UAV's current position. This repeated recomputation is correct but may become expensive as environment size and change frequency increase.

Reinforcement learning (RL) provides a different approach. Rather than recomputing a path through explicit graph search, an RL policy maps observations to movement actions after training. This can potentially reduce deployment-time computation and allow reactive behavior under changing conditions. However, learned policies may be suboptimal, may fail in some scenarios, and do not provide the same optimality guarantees as Dijkstra [3]–[5].

Before testing RL's adaptability under dynamic conditions, it is necessary to establish that the trained policy performs reasonably well when the environment does not change at all; a policy that cannot approach optimal performance on a static grid is not a meaningful comparison point once obstacles start moving. We therefore evaluate all methods twice: first on a static 40-scenario benchmark that establishes a baseline for path-cost and compute-time behavior, and then on the paired 50-scenario dynamic benchmark that is this paper's primary contribution.

A first version of this comparison used only Dijkstra's

algorithm, run with full-graph replanning on every obstacle change, as the sole classical baseline. That comparison is a valid but narrow test: it asks whether RL beats one particular, deliberately unoptimized classical implementation, not whether RL beats classical planning in general. A reader could reasonably ask whether the reported compute-time result was really about "search vs. learned policy" or simply about "un-pruned search vs. pruned search." This revision adds A* replanning, which uses the same graph and the same replanning trigger as Dijkstra but prunes its search with an admissible heuristic, as a second classical baseline. Because A* and Dijkstra are both optimal on this non-negative-weight grid, any cost difference between them should be zero, and any compute-time difference isolates the effect of heuristic pruning rather than the effect of "search vs. RL." This lets us separate two previously conflated questions: (1) does RL beat classical graph search at all, and (2) was the previous compute-time comparison handicapping the classical side by using an unnecessarily naive implementation.

This paper asks whether a dynamically trained RL policy can match or outperform classical replanning — both a naive and a heuristic-pruned variant — in a controlled UAV grid-routing environment. We compare all methods on success rate, path cost, and compute time, both when the environment is static and when it changes during flight.

The main contributions are:

- a shared 15×15 UAV grid environment with identical movement and dynamic-obstacle rules for all three methods;
- a static baseline evaluation validating the trained RL policy's path quality and compute-time behavior before dynamic obstacles are introduced, independently re-run and confirmed for this revision;
- two classical replanning baselines — naive Dijkstra and heuristic-pruned A* — that recompute after each dynamic obstacle change, isolating the effect of search-time optimization from the effect of using a learned policy at all;
- a Double DQN + HER policy trained through curriculum learning and evaluated on the same 50 scenario IDs and start-goal pairs as both classical baselines;
- paired statistical comparisons using Wilcoxon signed-rank testing across all three method pairs, plus Wilson confidence intervals on the small-sample success-rate comparisons;
- a full account of the RL policy's hyperparameters, training curriculum, and a documented single-seed limitation, so the reported numbers describe one trained policy rather than a distribution over training runs.

II. Related Work

Classical UAV path planning methods include graph-search algorithms, sampling-based planners, and potential-field approaches. Graph-search methods such as Dijkstra and A* are widely used because they are interpretable and provide strong guarantees when the environment is represented as a known graph with non-negative edge weights. A* [6] extends Dijkstra with an admissible heuristic that prunes the search toward the goal; on a non-negative-weight graph the two algorithms are guaranteed to return identically optimal paths, but A* typically expands far fewer nodes. Sampling-based planners such as RRT and PRM can explore larger spaces, but their randomized nature means they do not necessarily return optimal paths and can struggle in cluttered environments [1], [7]. This project uses grid-search planners rather than a sampling-based method as the classical baseline because they are deterministic, optimal on a static graph, and have well-characterized computational complexity, properties that make the trade-off being measured easier to state precisely. We deliberately include both Dijkstra and A* as separate baselines: Dijkstra isolates the cost of *naive* full-graph replanning, while A* isolates how much of that cost is attributable to the absence of heuristic pruning rather than to graph search itself. Incremental replanners such as D* Lite [8] go further still by reusing information from the previous search, and remain a natural next baseline beyond the scope of this paper.

Recent surveys of UAV path planning emphasize that classical planners remain important reference points because of their completeness, optimality, and predictable computational behavior [1]–[3]. Their main limitation in dynamic environments is that they generally assume a known graph. When obstacles or edge costs change, the planner must update or recompute a route.

Reinforcement learning has been explored as an alternative for UAV routing and obstacle avoidance. RL methods learn policies through interaction rather than explicit shortest-path search. Deep RL methods such as DQN are particularly relevant when the state space becomes too large for tabular Q-learning [4], [5], [9]. An improved Q-learning variant with revised reward and Q-value update rules has been shown to reduce computation time by 50% and shorten path length by 30% compared to a standard SARSA baseline on a UAV path-planning task [4]. Prior work reports that RL can improve autonomy and adaptability in complex settings, but it often trades away the guarantees of classical methods.

Dynamic and online UAV routing is especially relevant to this project. Rocha et al. propose dynamic Q-planning for online UAV path planning in unknown and complex environments, comparing reinforcement-learning-based planning with classical alternatives [10]. This supports the motivation for testing learning-based adaptation against a deterministic replanning baseline.

Much of the RL-versus-classical UAV literature compares methods across different metrics, environments, and hardware, making head-to-head comparison across papers difficult; few evaluate both approaches on the exact same graph, obstacle layout, and start-goal pairs with a paired statistical test on the difference. This project fits into the literature by closing that specific gap: a controlled same-scenario comparison rather than a broad benchmark. Both methods are evaluated on the same grid, same dynamic cells, and same 50 start-goal pairs. This paired design allows statistical testing of path-cost

and compute-time differences instead of relying on anecdotal comparisons.

III. Methodology

A. Environment Model

The UAV operating area is represented as a 15×15 grid. Each traversable cell is a node, and the UAV can move in eight directions: north, south, east, west, and the four diagonals. Orthogonal moves have cost 1.0, while diagonal moves have cost $\sqrt{2}$. All edge costs are non-negative, making Dijkstra's algorithm valid for shortest-path computation.

Coordinates are represented as (row, col), with $(0, 0)$ at the top-left corner. A movement is valid only if the destination cell is inside the grid and not blocked. Both Dijkstra and RL use the same movement rules to prevent inconsistencies between the two implementations.

B. Dynamic Obstacles

The dynamic environment uses fixed-position obstacles that toggle between blocked and passable states. For the paired dynamic evaluation, static obstacle density is set to 0.0 on both sides. This removes seeded static-layout mismatch between the Dijkstra and RL environments. The only changing cells are the three shared dynamic cells $(4, 4)$, $(8, 8)$, and $(12, 11)$.

Neither method receives advance visibility into future toggles. Both methods observe only the current grid state after a change occurs. This keeps the Dijkstra baseline genuinely naive and ensures that the RL policy is not given predictive information unavailable to the classical planner.

C. Dijkstra Replanning Baseline

The first classical baseline applies Dijkstra's algorithm to the current grid graph. In the dynamic case, whenever the environment changes, the planner recomputes a full shortest path from the UAV's current position to the goal. This naive replanning strategy is intentionally simple: it is correct and interpretable, but can pay the full graph-search cost multiple times during a route, expanding nodes in all directions regardless of where the goal lies.

With a binary heap priority queue, Dijkstra's algorithm has time complexity $O((V + E) \log V)$, where V is the number of free cells and E is the number of valid movement edges. In this grid, each cell has at most eight neighbors, so E grows approximately linearly with V.

D. A* Replanning Baseline

The second classical baseline applies A* [6] to the same grid graph, using the octile distance (the natural admissible heuristic for 8-connected grids with unit and $\sqrt{2}$ move costs) as the heuristic function. A* is triggered by exactly the same replanning events as Dijkstra — one initial plan plus one replan per dynamic obstacle toggle — and returns a path over the same graph with the same edge costs. The only difference from the Dijkstra baseline is that A* prioritizes expanding nodes that appear closer to the goal, so on a non-negative-weight grid the two algorithms are guaranteed to return paths of identical cost; A* simply reaches that answer by expanding fewer nodes. This baseline exists specifically to separate two effects that a single naive-Dijkstra comparison conflates: whether classical graph search is expensive because it is *search*, or expensive because it is *unpruned* search. Section V reports both the node-expansion count and the wall-clock time for A*, alongside the same metrics for Dijkstra, on the identical 50 dynamic scenarios.

E. Reinforcement Learning Policy

The RL method uses the same eight movement actions and the same grid, obstacle, and edge-cost rules as both classical baselines. The agent is a Double DQN [11] trained with Hindsight Experience Replay (HER) [12], implemented on top of Stable-Baselines3's DQN using a custom `SafeHerReplayBuffer` that forces the terminal-done flag when a HER-relabeled achieved goal coincides with the relabeled desired goal, and a custom training step that computes the Double DQN target (action selection from the online network, action evaluation from the target network) rather than the single-network max used by vanilla DQN [13]. Double DQN was adopted after an earlier vanilla-DQN + HER run exhibited Q-value overestimation (values around 46 against an expected scale near 2.0), which Double DQN, together with a smaller per-step reward magnitude, corrected.

Observation. The 61-dimensional observation concatenates: (i) the UAV's normalized row/col position and normalized row/col displacement to the goal (4 values); (ii) a flattened 7×7 local grid patch centered on the UAV, with out-of-bounds cells treated as obstacles (49 values); and (iii) a one-hot encoding of the previous action (8 values). The policy does not observe the full 15×15 grid, only this local window plus its own position relative to the goal.

Reward. The reward combines a sparse terminal term (+1.0 on reaching the goal, -1.0 on collision, -0.1 per step otherwise) with potential-based shaping [14], $\Phi(s) = -d(s, \text{goal})/d_{\max}$, where $d(\cdot, \cdot)$ is the shortest-path distance on the current grid (precomputed once per fixed grid via an all-pairs Dijkstra table, so that HER's relabeled compute_reward calls are $O(1)$ lookups rather than live searches) and $d_{\max}$ is the grid diagonal. Using graph distance rather than Euclidean distance for shaping ensures the agent is rewarded for obstacle-aware progress, not straight-line proximity that a wall may block.

Network and optimization. The Q-network is a two-layer MLP with 128 units per layer. Table I lists the full hyperparameter configuration.

Curriculum. Training proceeds in three phases on the identical fixed 15×15 grid (obstacle density 0.20, seed 42) used for the static evaluation: Phase 1 trains 300,000 timesteps with no dynamic obstacles, establishing basic static-grid competence with the reward and observation design above; Phase 2 fine-tunes the Phase 1 checkpoint for 100,000 further timesteps with a single mild dynamic obstacle at $(8, 8)$ toggling every 10 steps, with exploration re-opened to ε: $0.30 \rightarrow 0.05$ over 20% of the phase; Phase 3 fine-tunes the Phase 2 checkpoint for a

TABLE I
DQN + HER HYPERPARAMETERS (IDENTICAL ACROSS ALL CURRICULUM PHASES).

Hyperparameter	Value
Algorithm	Double DQN [11]
Replay strategy	HER, `future` strategy, k=4 [12]
Network architecture	MLP, 2 $\times$ 128 hidden units
Learning rate	1×10^{-3}
Replay buffer size	100,000 transitions
Batch size	256
Discount factor γ	0.99
Target update (τ, interval)	1.0 (hard), every 1,000 steps
Train frequency	every 2 environment steps
Gradient steps per update	1
Exploration (ε-greedy, Phase 1)	$1.0 \rightarrow 0.05$ over 50% of training
Reward: goal / collision / step	$+1.0$ / -1.0 / -0.1
Reward shaping	Potential-based, $\Phi = -d/d_{\max}$ [14]
Training seed	42 (single run; see Sec. VI-A)

final 100,000 timesteps on the full three-obstacle configuration at $(4, 4)$, $(8, 8)$, $(12, 11)$ toggling every 5 steps, matching the evaluation configuration in Section IV. The reported dynamic-evaluation policy is therefore the Phase 3 checkpoint after 500,000 cumulative timesteps of training. Unlike Dijkstra and A*, the RL policy does not explicitly search the graph at evaluation time; it performs a single forward pass through the trained network per step.

F. Compute Time Measurement

All three methods report compute time as the total decision-making time required to traverse a full route, not per-decision latency. For Dijkstra and A*, this is the cumulative wall-clock time across all planning calls made during a run: one initial call plus one call per dynamic change event, using the identical replanning trigger for both planners so that any timing difference reflects the search algorithm itself rather than a different number of replans. For the RL policy, this is the sum of wall-clock time for every `.predict()` call made during an episode; environment stepping, reward computation, and observation construction are excluded from the timer.

Because the classical planners resolve a route in one or a small number of full graph searches while the RL policy makes one lightweight forward pass per step, the reported totals are comparable only as route-level decision cost: how much compute each method consumed to get from start to goal in that scenario, not as a per-decision latency. A single warm-up inference call is discarded before timing begins to exclude first-call framework initialization overhead from the RL measurements. We reiterate the caveat from the original study: millisecond-scale timings on a shared, non-dedicated evaluation machine are sensitive to implementation details, language-level overhead (both planners are pure Python; the RL policy runs through the PyTorch/Stable-Baselines3 stack), and hardware, and should be read as an internally consistent *relative* comparison between methods rather than an absolute claim about algorithm speed in general.

IV. Experimental Setup

A. Static Evaluation

The static evaluation uses 40 paired start-goal scenarios on the same 15×15 grid, with obstacle density 0.2 and seed 42. Start-goal pairs are drawn from the free cells of a single `GridEnvironment` instance and validated against Dijkstra on that same grid; the DQN evaluation environment receives the identical obstacle layout via direct injection rather than independent seeding. No dynamic obstacles are present in this evaluation. For this revision, the static evaluation script was re-run independently to confirm the previously reported success-rate and path-cost figures, and was additionally instrumented to measure route-level compute time directly (Sec. III-F), since the original run did not persist raw per-scenario timing data.

B. Dynamic Evaluation

The paired dynamic evaluation uses 50 matched scenarios. Each scenario has a fixed scenario ID, start cell, and goal cell. The same scenario list is used for Dijkstra, A*, and RL evaluation. Pairing is important because comparing unrelated random episodes would confound method performance with scenario difficulty.

The evaluation configuration is:

- grid size: 15×15;
- movement: 8-connected;
- orthogonal movement cost: 1.0;
- diagonal movement cost: $\sqrt{2}$;
- static obstacle density: 0.0;
- dynamic cells: $(4, 4)$, $(8, 8)$, and $(12, 11)$, each toggling on a 5-step period;
- number of scenarios: 50, shared across all three methods.

The measured metrics are success rate, realized dynamic path cost, compute time, replan/node-expansion counts (for the two classical planners), and paired path-cost and compute-time differences. Statistical significance on continuous paired metrics (path cost, compute time) is evaluated using the two-sided Wilcoxon signed-rank test on paired successful scenarios. Because the success-rate comparisons involve small samples (50 or 49 scenarios), we additionally report 95% Wilson score confidence intervals [15] on each method's success proportion, which are more reliable than a normal approximation at this sample size and do not require a paired design.

C. Single-Seed Training Caveat

All RL results in this paper — static and dynamic — come from one trained policy (training seed 42, Table I). Neural-network RL training is known to be sensitive to random seed, and repeating training with a different seed can shift success rate, path-cost gap, and even qualitative behavior [16]. We did not have compute or time budget in this revision to retrain multiple seeds and report variance, and we treat this as an open limitation (Sec. VI-A) rather than implying the reported numbers characterize DQN+HER's performance ceiling or typical variance on this task. The classical baselines (Dijkstra, A*) are deterministic and have no equivalent seed sensitivity.

V. Results

A. Week 2: Static Environment

The static evaluation used 40 paired start-goal scenarios on a 15×15 grid with obstacle density 0.20 and seed 42. Start-goal pairs were drawn from the free cells of a single `GridEnvironment` instance and validated against Dijkstra on that same grid. The DQN evaluation environment had the same 15×15 obstacle layout as the Dijkstra environment, achieved by directly copying `simra_env.blocked` into the DQN grid after construction; this is a stronger guarantee than matching seeds, as it carries no dependency on RNG implementation details agreeing between the two environment classes. Dijkstra computed a single shortest-path query per scenario; the DQN policy ran a full episode from each start to goal.

TABLE II
Static Evaluation Summary (40 scenarios). Compute-time figures were re-measured for this revision on the evaluation machine used throughout this paper; they are of the same order as, but not numerically identical to, the originally reported figures, consistent with the hardware sensitivity noted in Sec. III-F.

Metric	Dijkstra	DQN
Success rate	40/40 (100%)	39/40 (97.5%)
Mean path cost	—	+0.9248 above Dijkstra
Max suboptimality gap	—	+3.6569
Mean compute time (all scenarios)	0.72 ms	3.86 ms
Mean compute time (successes only)	0.72 ms	2.40 ms

DQN matched Dijkstra exactly (gap = 0) on 14 of 39 successful scenarios and produced a higher path cost on the remaining 25. The single failure (start $(12, 0)$, goal $(13, 10)$) was a step-limit timeout at 225 steps with no collision, indicating the policy entered a loop rather than crashing. The compute-time figures are route-level totals as defined in Section III-F: one Dijkstra call per scenario versus accumulated `.predict()` calls per DQN episode. Both the success/path-cost figures and the compute-time figures in Table II were produced by an independent re-run of the static evaluation for this revision (Sec. IV); the success-rate and path-cost numbers matched the original run exactly, which is expected since both Dijkstra and the loaded DQN checkpoint are deterministic at evaluation time (`deterministic=True` action selection), while the compute-time numbers were newly measured rather than reused. On this re-run, DQN was on average $3.3\times$ slower than Dijkstra on scenarios where DQN succeeded, and $5.4\times$ slower when the one timeout episode — which pays the full 225-step budget of forward passes before terminating — is included.

B. Dynamic Environment

The dynamic evaluation used 50 paired start-goal scenarios sharing the same scenario IDs and start-goal pairs across all three methods. Static obstacle density was set to 0.0 on all sides; the only changing cells were the three shared dynamic obstacles at $(4, 4)$, $(8, 8)$, and $(12, 11)$, each toggling on a five-step period. Dijkstra and A* both succeeded in all 50 scenarios; the RL policy succeeded in 49 of 50. The failed RL scenario is included in success-rate reporting but excluded from paired path-cost and compute-time statistics because no valid RL path cost exists for it, leaving $n = 49$ scenarios where all three methods succeeded for the paired comparisons below.

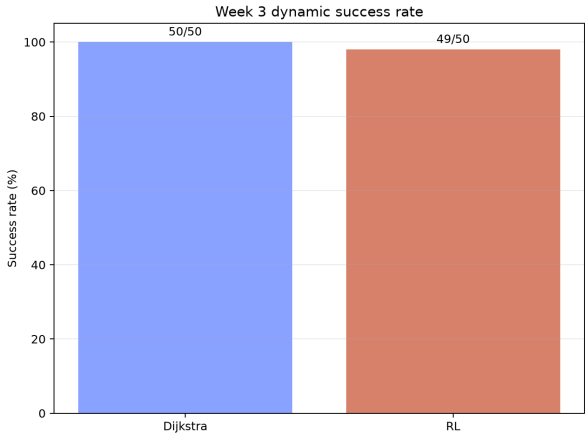

Fig. 1. Dynamic-routing success rate over 50 matched scenarios.

1) Success Rate: Figure 1 shows the success rate for Dijkstra and RL as originally reported. Table III adds A* and reports 95% Wilson confidence intervals for all three methods given the small sample size. Dijkstra and A* both achieved 100% success (50/50); RL achieved 98% (49/50). The confidence intervals overlap substantially at this sample size — with only 50 trials, a single-scenario difference in outcome is not enough to separate 100% from 98% with strong confidence on its own. The success-rate comparison should therefore be read alongside the path-cost comparison below, where the sample is larger in effect (49 paired, non-tied observations) and the statistical evidence is much stronger.

TABLE III
Dynamic-evaluation success rate with 95% Wilson confidence intervals (50 scenarios each).

Method	Success rate	95% Wilson CI
Dijkstra	50/50 (100.0%)	[92.9%, 100.0%]
A*	50/50 (100.0%)	[92.9%, 100.0%]
RL (DQN+HER)	49/50 (98.0%)	[89.5%, 99.6%]

2) Path Cost: Path-cost statistics are computed on the 49 scenarios where both methods succeeded. Dijkstra was never more expensive than RL: RL matched Dijkstra in 16 scenarios and produced a higher path cost in 33 scenarios. The mean path-cost difference, measured as RL minus Dijkstra, was +1.369441, and the median difference was +0.828427.

A Wilcoxon signed-rank test on paired path-cost differences gave $W = 0$ and $p = 5.64 \times 10^{-7}$. This result is statistically significant, indicating that Dijkstra produced significantly shorter or equal paths than RL in the tested dynamic environment.

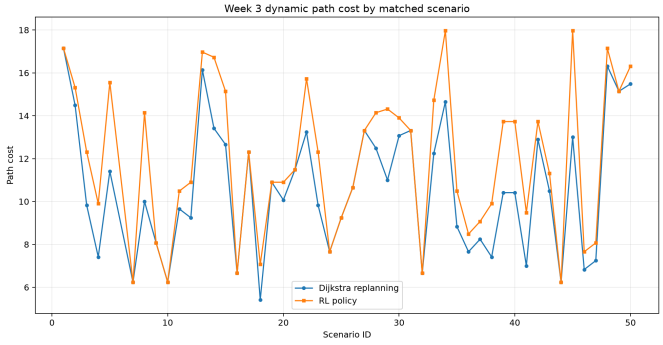

Fig. 2. Dynamic path cost by matched scenario for Dijkstra replanning and RL.

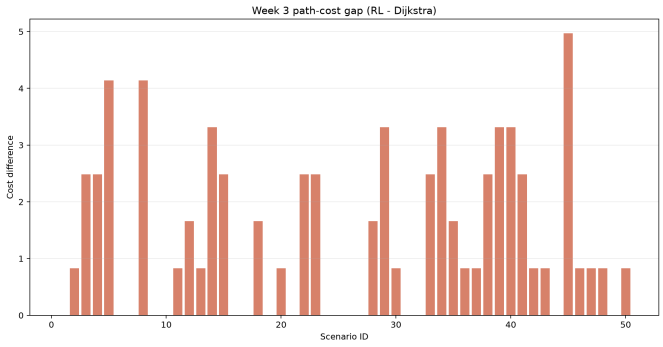

Fig. 3. Path-cost gap measured as RL minus Dijkstra. Positive values indicate higher RL path cost.

Because A* is optimal on this non-negative-weight graph, its dynamic path cost was identical to Dijkstra's on all 49 jointly-successful scenarios (mean and maximum absolute difference both 0.000000), exactly as predicted by the algorithms' shared optimality guarantee. Consequently RL's path-cost gap against A* is numerically identical to its gap against Dijkstra (Wilcoxon $W = 0$, $p = 5.23 \times 10^{-7}$, computed independently on the A*-paired subset). This identity is itself a useful sanity check: it confirms that A* was implemented and evaluated correctly against the same graph and edge costs as Dijkstra, and that the RL policy's cost disadvantage is a genuine gap against *optimal* planning, not an artifact of which classical planner happens to be used as the reference.

3) Compute Time: Mean cumulative Dijkstra planning time was 4.628 ms, while mean RL evaluation time was 5.334 ms on the paired successful scenarios ($n = 49$). The Wilcoxon p-value for this compute-time difference was 0.063, which is not statistically significant at $p < 0.05$ — the finding reported in the original evaluation.

Adding A* changes this picture substantially. A* resolved the same replanning events — an identical mean of 2.45 replans per scenario, since both classical planners react to the same dynamic-obstacle triggers — in a mean of 0.288 ms, versus Dijkstra's 4.628 ms: roughly **16× faster**, driven by A*'s heuristic pruning reducing mean node expansions per scenario to 31.7 out of the grid's 225 cells, compared

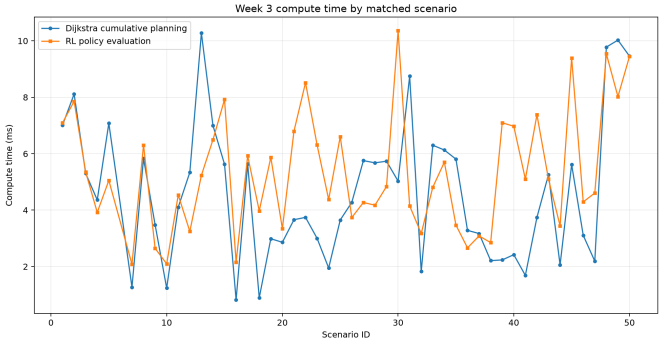

Fig. 4. Compute time by matched scenario. Dijkstra time is cumulative replanning time; RL time is policy evaluation time.

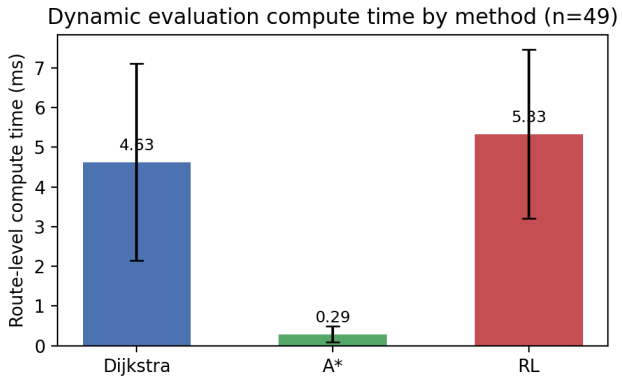

Fig. 5. Mean route-level compute time by method on the 49 jointly-successful dynamic scenarios (error bars: 1 std. dev.). A* resolves the same replanning events as Dijkstra roughly 16× faster at identical path cost.

to Dijkstra's exhaustive expansion. A paired Wilcoxon test on A* vs. Dijkstra compute time gives $p = 3.6 \times 10^{-15}$, and A* vs. RL compute time gives the same order of significance ($p = 3.6 \times 10^{-15}$, A* faster). Figure 5 places all three methods on the same axis.

This reframes the original compute-time conclusion. The original paper's non-significant $p = 0.063$ result compared RL only against *naive* Dijkstra and concluded that "the anticipated compute-time advantage of RL did not reach significance... likely because the grid is small enough that repeated Dijkstra replanning remains cheap." That explanation undersold the classical side: a large part of naive Dijkstra's cost is not an inherent property of graph-search replanning at this grid size, but a consequence of not pruning the search at all. Once a standard, easy-to-implement heuristic is added, the classical planner becomes decisively faster than both naive Dijkstra *and* RL, while still returning optimal paths. RL's per-step forward-pass cost, which the original discussion framed as a potential deployment-time advantage over repeated graph search, does not materialize as an advantage against either classical baseline at this grid size — it is the slowest of the three methods in route-level compute time.

C. Generalization Probe

The evaluations above all use the same fixed grid layout (seed 42) that the RL policy was trained on, since `fixed_grid=True` was used throughout training to make HER's reward computation tractable (Sec. III-E). This means the reported RL success rates describe performance on a grid the policy has effectively memorized the structure of, not generalization to unseen layouts. As an informal robustness probe conducted during development, the Phase-3 checkpoint was additionally evaluated on 50 episodes drawn from a single unseen obstacle layout (seed 999, not used anywhere in training or in the reported evaluations), with random start/goal pairs rather than the fixed 50-scenario protocol used elsewhere in this paper. The agent reached the goal in 47 of 50 episodes (94%), with zero collisions and three step-limit timeouts. Trace inspection of the three timeouts showed the agent navigating correctly for most of each episode before entering a two-cell oscillation after being structurally cut off from a direct route by an obstacle configuration not present in the training grid; reward magnitudes during these episodes were small and correctly scaled (-0.02 to -0.17 per step), which rules out the earlier Q-value overestimation bug (Sec. III-E) as the cause and points instead to a pure generalization gap from single-layout training.

We report this result as a directional signal, not a rigorous generalization benchmark: it uses one unseen seed rather than a distribution of layouts, was not paired against Dijkstra or A* on the same seed, and predates some of the reward-shaping changes reflected in Table I. We include it because it is the only evidence in this project bearing on out-of-distribution behavior, and because the failure mode it surfaces — confident local navigation followed by oscillation rather than exploratory replanning when structurally blocked — is a more informative limitation than a bare success-rate number would be. A rigorous multi-layout generalization benchmark, ideally with `fixed_grid=False` training as already scoped in the project's internal notes, is future work (Sec. VI-A).

VI. Discussion

The results show that classical replanning remains the stronger approach on the current controlled benchmark, and that this conclusion is more robust than the original single-baseline comparison suggested. Both Dijkstra and A* succeeded on every scenario and produced shorter or equal paths than RL on all paired successful scenarios; because Dijkstra and A* are both optimal on this graph, RL's path-cost disadvantage is a gap against *optimality itself*, not an artifact of which classical planner was chosen as the point of comparison. The RL policy still demonstrates substantial adaptability, completing 49 out of 50 dynamic scenarios under the same obstacle schedule and scenario pairing. However, these results do not support a claim that RL outperforms classical replanning in this setup, on either path quality or reliability.

The compute-time result is where adding A* changes the paper's conclusion rather than just reinforcing it. The original study found no significant compute-time difference between RL and naive Dijkstra and attributed this to grid size: "a 15×15 graph is small enough that repeated Dijkstra replanning remains cheap." That framing implicitly treated Dijkstra as representative of "classical graph search" in general. It is not. A* solves the identical replanning problem, with the identical optimality guarantee, roughly $16\times$ faster than Dijkstra by pruning search toward the goal instead of expanding uniformly outward (Sec. V-B). A common motivation for RL in this literature is that a trained policy may avoid repeated graph search during deployment; here, RL was in fact the *slowest* of the three methods on route-level compute time, once a properly pruned classical planner is included. This does not mean RL can never win on compute time — a heuristic search's cost still scales with how much of the grid must be explored before the goal is found, and larger or more cluttered environments could favor a constant-cost forward pass over search of any kind — but it does mean the earlier conclusion ("RL's expected computational advantage may only become visible on larger grids") was drawn against an unnecessarily weak classical opponent, and should be re-tested against A* or an incremental planner such as D* Lite [8] before being treated as settled.

The study therefore supports a trade-off framing, sharpened from the original: a well-implemented classical planner provides optimality, reliability, *and* the faster route-level compute time at this grid size; RL provides policy-based adaptation without an explicit graph search, but its learned decisions are both slower and less optimal than either classical baseline at the tested scale, and its one failure represents a reliability gap that neither classical planner exhibited. The current results favor classical planning at this scale while leaving open, as a genuinely untested question rather than a mild caveat, whether RL's balance of tradeoffs improves as environment size, obstacle density, or replanning frequency increase to the point where even a pruned search becomes expensive.

A. Limitations

This study has several limitations, some carried over from the original evaluation and some specific to this revision.

Scale is a deliberate, acknowledged design choice, not an oversight. The 15×15 grid, zero static obstacle density in the dynamic evaluation, and three fixed toggle cells were chosen to remove confounds — to guarantee that Dijkstra and RL (and, in this revision, A*) are compared on literally the same graph, the same replanning triggers, and the same start/goal pairs, so that any measured difference is attributable to the planning method rather than to incidental differences in problem difficulty. This buys internal validity at the cost of external validity: the benchmark is small and clean, not representative of a cluttered, large-scale UAV routing domain. Readers should treat the paper's claims as holding *on this controlled benchmark*, not as claims about UAV routing at operational scale.

Single training seed. All RL results come from one trained policy (seed 42; Sec. IV). We did not retrain with multiple seeds to report variance, and RL training is known to be

seed-sensitive [16]: a different seed could plausibly change the success rate, the size of the path-cost gap, or the identity of the failing scenario, though a shift large enough to reverse the paper's central Wilcoxon result ($p = 5.64 \times 10^{-7}$ against Dijkstra, $n = 49$) would require a substantially different policy. We report this as an open limitation rather than treating the single run as characterizing DQN+HER's typical behavior or performance ceiling on this task.

No ablations. The curriculum (Sec. III-E) and the Double DQN + potential-based-shaping design were both adopted after diagnosing specific failures during development (Q-value overestimation under vanilla DQN; see Sec. III-E), but this paper does not report a controlled ablation — e.g., training without curriculum, or without HER — that would isolate how much each design choice contributes to the final result. The development history that motivated these choices is documented in the project repository but not formally re-run as a paired experiment here.

Limited generalization evidence. All formal evaluations use the single fixed training grid. The informal generalization probe in Sec. V-C (94% success on one unseen seed) is suggestive but not a rigorous benchmark, and surfaces a specific, unresolved failure mode — oscillation when structurally blocked in an unfamiliar layout — that the fixed-grid evaluations in this paper cannot detect at all.

Remaining scope limitations. The RL policy has one failed scenario, which should be treated as a real limitation rather than ignored. Compute-time measurements at millisecond scale remain sensitive to implementation details, language-level overhead, and hardware (Sec. III-F); the relative ordering among the three methods was stable across the original and re-measured runs, but the absolute figures should not be treated as portable to other hardware. The dynamic setup still uses only three fixed toggle cells rather than dense, moving, or stochastic obstacles, wind fields, or multi-agent airspace constraints.

Future work should evaluate larger grids, reintroduce matched static obstacles, vary dynamic obstacle frequency, compare against an incremental replanner such as D* Lite or Lifelong Planning A* (which, unlike Dijkstra and A* here, reuse information from the previous search rather than replanning from scratch), retrain RL across multiple seeds to report variance, run the curriculum and HER ablations motivated above, and evaluate with `fixed_grid=False` training to test generalization formally. A richer benchmark incorporating energy budgets, no-fly zones, and partial observability would also better reflect real UAV routing constraints.

VII. Conclusion

This paper compared two classical replanning baselines — naive Dijkstra and heuristic-pruned A* — against a Double DQN + HER policy trained with curriculum learning, for UAV routing in a shared dynamic grid environment. The evaluation used 50 matched scenarios with identical start-goal pairs and shared dynamic obstacles across all three methods. Dijkstra and A* both succeeded on all 50 scenarios and, as guaranteed by their shared optimality on this graph, produced identical path costs; RL succeeded on 49. On paired successful scenarios, both classical planners produced significantly shorter or equal paths than RL ($W = 0$, $p = 5.64 \times 10^{-7}$ vs. Dijkstra; $p = 5.23 \times 10^{-7}$ vs. A*).

Adding A* changed the compute-time conclusion, not just extended it. Naive Dijkstra showed no statistically significant compute-time advantage over RL ($p = 0.063$), which the original evaluation attributed to the grid being small enough that repeated full-graph replanning stays cheap. A* resolved the identical replanning problem roughly $16\times$ faster than Dijkstra ($p = 3.6 \times 10^{-15}$) by pruning its search with an admissible heuristic, while returning exactly the same path costs. RL was the slowest of the three methods in route-level compute time. The earlier explanation — that RL's compute-time advantage might simply be latent at this scale — was measuring RL against an unnecessarily naive classical opponent; a properly implemented classical planner is both more optimal and faster than RL here, not merely more optimal.

On the static 40-scenario evaluation, independently re-run for this revision, Dijkstra succeeded on every scenario while DQN succeeded on 97.5%, with a mean path-cost penalty of 0.92 units above optimal and a re-measured $\sim 3.3\times$ compute-time overhead for DQN on successful scenarios, consistent with the original study's static findings.

For the current controlled 15×15 setup, classical replanning — especially once heuristic pruning is used — remains superior to the tested RL policy in reliability, path optimality, and compute cost. The RL policy demonstrates meaningful adaptability but does not outperform either classical baseline on any measured axis at this scale. This conclusion should be read alongside two explicit caveats this revision adds rather than resolves: all RL results come from a single trained policy (one random seed, no variance estimate), and the benchmark's small, clean scale was a deliberate methodological choice to maximize internal validity, not a claim about performance at operational UAV scale. Future work should test whether RL's balance of tradeoffs improves in larger, denser, or more frequently changing environments, against an incremental classical planner such as D* Lite, and across multiple RL training seeds.

References

[1] S. Ghambari, M. Golabi, L. Jourdan, J. Lepagnot, and L. Idoumghar, "Uav path planning techniques: a survey," *RAIRO - Operations Research*, vol. 58, no. 4, pp. 2951–2989, 2024.

[2] N. Johnson, S. Shafaei, A. Karem, and S. Sarkar, "A survey of risk-calibrated certifiably safe and resource-aware (rcsr) path planning for unmanned aerial vehicles," *Drones*, vol. 10, no. 5, p. 351, 2026.

[3] A. Mannan, M. S. Obaidat, K. Mahmood, A. Ahmad, and R. Ahmad, "Classical versus reinforcement learning algorithms for unmanned aerial vehicle network communication and coverage path planning: A systematic literature review," *International Journal of Communication Systems*, vol. 36, no. 5, p. e5423, 2023.

[4] G.-T. Tu and J.-G. Juang, "Uav path planning and obstacle avoidance based on reinforcement learning in 3d environments," *Actuators*, vol. 12, no. 2, p. 57, 2023.

[5] Y. Guo and Z. Liu, "Uav path planning based on deep reinforcement learning," *International Journal of Advanced Network, Monitoring and Controls*, vol. 8, no. 3, pp. 81–88, 2023.

[6] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[7] S. Venu and M. Gurusamy, "A comprehensive review of path planning algorithms for autonomous navigation," *Results in Engineering*, vol. 28, p. 107750, 2025.

[8] S. Koenig and M. Likhachev, "Fast replanning for navigation in unknown terrain," *IEEE Transactions on Robotics*, vol. 21, no. 3, pp. 354–363, 2005.

[9] J. Zhao, Z. Gan, J. Liang, C. Wang, K. Yue, W. Li, Y. Li, and R. Li, "Path planning research of a uav base station searching for disaster victims' location information based on deep reinforcement learning," *Entropy*, vol. 24, no. 12, p. 1767, 2022.

[10] L. G. S. d. Rocha, K. A. Q. Caldas, M. H. Terra, F. Ramos, and K. C. T. Vivaldini, "Dynamic q-planning for online uav path planning in unknown and complex environments," 2024.

[11] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," in *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, 2016, pp. 2094–2100.

[12] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba, "Hindsight experience replay," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[13] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[14] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 1999, pp. 278–287.

[15] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[16] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.